

Lab Exercise 2- Implementing JWT Authentication in Java API and Testing Using Postman

1. Aim

To implement **JSON Web Token (JWT)** authentication in a **Java REST API** using VS Code and test the API endpoints using Postman.

2. Software Requirements

- Java JDK 17
 - VS Code
 - Java Extension Pack (VS Code)
 - Maven
 - Postman
-

3. Theory

JSON Web Token (JWT) is used to securely transmit information between a client and server.

JWT Authentication Flow:

1. User sends **login credentials**
2. Server verifies credentials
3. Server generates **JWT token**

4. Client stores token
 5. Client sends token in **Authorization header**
 6. Server validates token and returns data
-

4. Procedure

Step 1: Create Spring Boot Project in VS Code

Open **VS Code**

Press:

Ctrl + Shift + P

Select:

Spring Initializr: **Create a Maven Project**

Choose:

Option	Value
Language	Java
Group	com.example
Artifact	jwtapi
Packaging	Jar
Java Version	17

Dependencies:

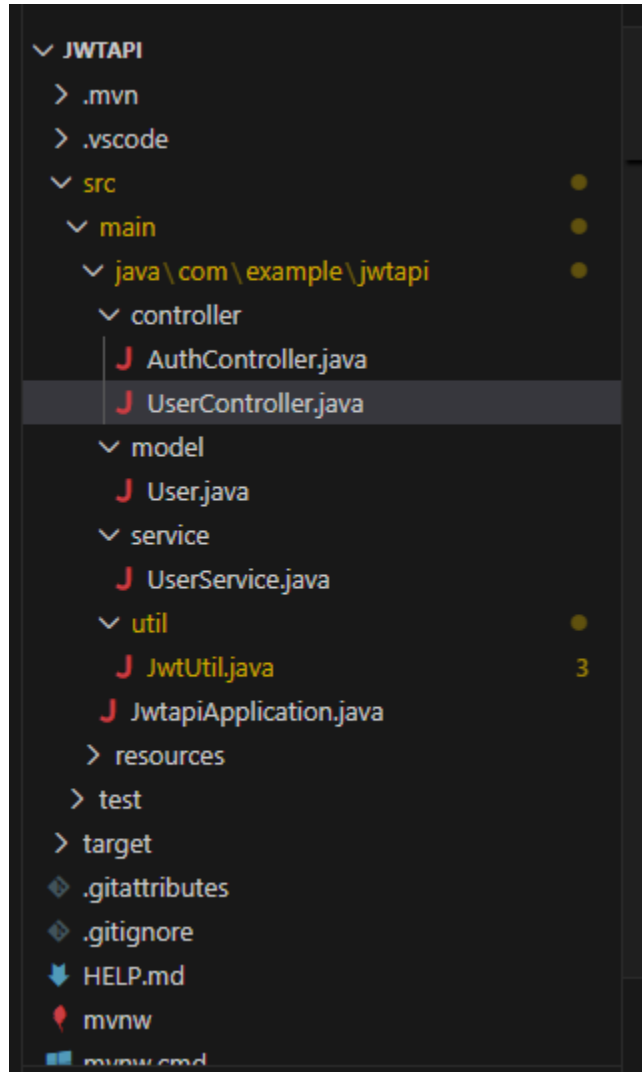
Spring Web

Step 2: Open the Project Folder

Open the generated folder:

jwtapi

Project Structure



4. pom.xml (JWT Dependency)

Add these dependencies.

```
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
  <version>0.11.5</version>
</dependency>

<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-impl</artifactId>
  <version>0.11.5</version>
  <scope>runtime</scope>
</dependency>

<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-jackson</artifactId>
  <version>0.11.5</version>
  <scope>runtime</scope>
</dependency>
```

Explanation

These libraries allow the project to:

- Generate JWT tokens
 - Validate JWT tokens
 - Parse token data
-

5. Main Application File

File:

```
JwtapiApplication.java
```

```
package com.example.jwtapi;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class JwtapiApplication {

    public static void main(String[] args) {
        SpringApplication.run(JwtapiApplication.class, args);
    }
}
```

Explanation

This is the **entry point of the Spring Boot application**.

@SpringBootApplication enables:

- Auto configuration
- Component scanning
- Embedded server

When executed it starts the **Spring Boot server (Tomcat)**.

6. User Model

File:

model/User.java

```
package com.example.jwtapi.model;

public class User {

    private String username;
    private String password;

    public String getUsername() {
        return username;
    }

    public void setUsername(String username) {
        this.username = username;
    }

    public String getPassword() {
        return password;
    }

    public void setPassword(String password) {
        this.password = password;
    }
}
```

Explanation

This class represents the **user login request body**.

Example JSON received:

```
{  
  "username": "admin",  
  "password": "1234"  
}
```

Spring automatically converts JSON → Java object.

7. User Service

```
service/UserService.java
```

File:

```
package com.example.jwtapi.service;  
  
public class UserService {  
  
    public boolean validateUser(String username, String password) {  
  
        if(username.equals("admin") && password.equals("1234")) {  
            return true;  
        }  
  
        return false;  
    }  
}
```

Explanation

This class performs **basic user authentication**.

For simplicity the credentials are hardcoded:

```
username = admin  
password = 1234
```

In real applications this would check a **database**.

7. JWT Utility Class

File:

```
util/JwtUtil.java
```

```
package com.example.jwtapi.util;  
  
import io.jsonwebtoken.Jwts;  
import io.jsonwebtoken.SignatureAlgorithm;  
import java.util.Date;  
  
public class JwtUtil {  
  
    private static final String SECRET_KEY =  
        "myverysecuresecretkeymyverysecuresecretkey";  
  
    public static String generateToken(String username) {
```



```

return Jwts.builder()
    .setSubject(username)
    .setIssuedAt(new Date())
    .signWith(SignatureAlgorithm.HS256, SECRET_KEY.getBytes())
    .compact();
}

public static boolean validateToken(String token) {

    try {

        Jwts.parser()
            .setSigningKey(SECRET_KEY.getBytes())
            .parseClaimsJws(token);

        return true;

    } catch (Exception e) {
        return false;
    }
}
}

```

Explanation

This class handles **JWT operations**.

Functions:

Method	Purpose
generateToken	creates a JWT token

Method	Purpose
validateToken	verifies token validity

The **secret key must be at least 256 bits** for HS256 algorithm.

9. Authentication Controller

File:

```
controller/AuthController.java
```

```
package com.example.jwtapi.controller;

import com.example.jwtapi.model.User;
import com.example.jwtapi.service.UserService;
import com.example.jwtapi.util.JwtUtil;

import org.springframework.web.bind.annotation.*;

@RestController
public class AuthController {

    UserService service = new UserService();

    @PostMapping("/login")
    public String login(@RequestBody User user) {

        if(service.validateUser(user.getUsername(), user.getPassword())) {

            return JwtUtil.generateToken(user.getUsername());
        }

        return "Invalid credentials";
    }
}
```

```
}  
}
```

Explanation

This controller handles **login requests**.

Flow:

- 1 User sends username/password
 - 2 Server validates credentials
 - 3 If correct → generate JWT token
 - 4 Token returned to client
-

10. Protected Controller

File:

```
controller/UserController.java
```

```
package com.example.jwtapi.controller;  
  
import com.example.jwtapi.util.JwtUtil;  
  
import org.springframework.web.bind.annotation.*;  
import jakarta.servlet.http.HttpServletRequest;  
  
@RestController  
public class UserController {
```

```
@GetMapping("/profile")
public String profile(HttpServletRequest request) {

    String header = request.getHeader("Authorization");

    if(header == null || !header.startsWith("Bearer ")) {
        return "Token missing";
    }

    String token = header.substring(7);

    if(!JwtUtil.validateToken(token)) {
        return "Invalid token";
    }

    return "Welcome to the protected profile API";
}
```

Explanation

This API is **protected**.

Steps performed:

- 1 Extract Authorization header
 - 2 Check if token exists
 - 3 Extract JWT token
 - 4 Validate token
 - 5 If valid → allow access
-

11. Running the Project

In VS Code terminal:

```
mvn spring-boot:run
```

Server starts:

```
http://localhost:8080
```

12. Testing with Postman

Step 1 Login

POST

```
http://localhost:8080/login
```

Body JSON

```
{  
  "username": "admin",  
  "password": "1234"  
}
```

Response:

JWT_TOKEN

Example:

```
eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJhZG1pbiIsImhhdCI6MTc3MzE2MjkoMno.yWZBXs  
8mwZ2PO82DVhbOJG84qYYt_d7oKF3YzJojDHo
```

Step 2 Access Protected API

GET

http://localhost:8080/profile

Header

Authorization : Bearer JWT_TOKEN

Response

Welcome to the protected profile API

13. Expected Result

- JWT token generated successfully
 - Token validated before accessing protected API
 - Unauthorized requests blocked
-

14. Conclusion

The experiment successfully demonstrated **JWT authentication in a Java REST API**, including token generation and validation, and verified using Postman.